

Objetivo

Aplicar una metodología sistemática de troubleshooting sobre recursos Kubernetes. Diagnosticar Pods con ImagePullBackOff, CrashLoopBackOff y Pending. Identificar fallos de Services provocados por selectors, targetPort y probes incorrectas. Validar y corregir problemas de resolución DNS dentro del clúster. Utilizar kubectl describe, logs, events, exec y Pods temporales de diagnóstico.

Requisitos previos

- Haber completado las prácticas anteriores de Pods, Deployments, Services y DNS.
- Tener Docker Desktop, Minikube y kubectl disponibles.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Conservar el clúster Minikube de tres nodos utilizado en prácticas anteriores.
- Comprender estados como Pending, Running, ImagePullBackOff y CrashLoopBackOff.

Duración estimada

80 minutos

Introducción

En esta práctica recibirás ocho fallos intencionales distribuidos entre Pods, Services y DNS. No se trata únicamente de ejecutar correcciones; deberás observar síntomas, identificar evidencia en Events, logs y configuración, determinar la causa raíz y validar que la solución realmente restableció el comportamiento esperado.

Instrucciones

Preparación del laboratorio

Crearás todos los escenarios dentro de un namespace dedicado para evitar interferencias con prácticas anteriores.

Preparar el entorno

- **Paso 1.** Abre **Docker Desktop** y confirma que Minikube continúa activo antes de desplegar escenarios defectuosos.

- **Paso 2.** Abre **Visual Studio Code**, selecciona **Git Bash** como terminal integrada y crea el directorio del laboratorio.

```
mkdir -p /c/LABS/kubernetes/lab8  
cd /c/LABS/kubernetes/lab8
```

 Copiar

- **Paso 3.** Comprueba que los nodos continúan disponibles y que CoreDNS se encuentra operativo antes de comenzar.

```
kubectl get nodes  
kubectl get pods -n kube-system -l k8s-app=kube-  
dns
```

 Copiar

Salida esperada aproximada:

NAME	STATUS	ROLES
minikube	Ready	control-plane
minikube-m02	Ready	<none>
minikube-m03	Ready	<none>

 Copiar

NAME	READY	STATUS
coredns-<hash>-<id>	1/1	Running

Los tres nodos deben estar **Ready** y al menos un Pod de CoreDNS debe permanecer disponible.

- **Paso 4.** Crea **troubleshooting-lab** para aislar los ocho escenarios defectuosos de los recursos utilizados en prácticas anteriores.

```
kubectl create namespace troubleshooting-lab
```

 Copiar

Salida esperada:

```
namespace/troubleshooting-lab created
```

 Copiar

- **Paso 5.** Configura el namespace como predeterminado del contexto actual para reducir errores durante las operaciones de diagnóstico.

```
kubectl config set-context --current --  
namespace=troubleshooting-lab
```

 Copiar

Salida esperada aproximada:

```
Context "<nombre-del-contexto>" modified.
```

 Copiar

Crear los escenarios de fallo

- **Paso 6.** Crea el archivo `setup-failures.yaml`, que contendrá los ocho escenarios intencionalmente defectuosos.

```
touch setup-failures.yaml
```

 Copiar

- **Paso 7.** Agrega el contenido siguiente.

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-a1-imagepull
  namespace: troubleshooting-lab
  labels:
    scenario: a1
spec:
  containers:
    - name: app
      image: nginx:this-tag-does-not-exist-99999
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-a2-crashloop
  namespace: troubleshooting-lab
  labels:
    scenario: a2
spec:
  containers:
    - name: app
      image: nginx:1.27-alpine
      command: ["/bin/sh", "-c", "comando-
inexistente --flag"]
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-a3-pending
  namespace: troubleshooting-lab
  labels:
    scenario: a3
spec:
  containers:
    - name: app
      image: nginx:1.27-alpine
      resources:
        requests:
          cpu: "999"
          memory: "999Gi"
---
apiVersion: v1
kind: Pod
```

 Copiar

```
metadata:
  name: pod-b1-backend
  namespace: troubleshooting-lab
  labels:
    app: backend-real
spec:
  containers:
    - name: app
      image: nginx:1.27-alpine
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: svc-b1-wrong-selector
  namespace: troubleshooting-lab
spec:
  selector:
    app: backend-typo
  ports:
    - port: 80
      targetPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-b2-webserver
  namespace: troubleshooting-lab
  labels:
    app: webserver-b2
spec:
  containers:
    - name: app
      image: nginx:1.27-alpine
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: svc-b2-wrong-port
  namespace: troubleshooting-lab
spec:
  selector:
```

```
    app: webserver-b2
  ports:
    - port: 80
      targetPort: 9999
  ---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deploy-b3-liveness
  namespace: troubleshooting-lab
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app-b3
  template:
    metadata:
      labels:
        app: app-b3
    spec:
      containers:
        - name: app
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          livenessProbe:
            httpGet:
              path: /healthz-inexistente
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 5
            failureThreshold: 2
  ---
apiVersion: v1
kind: Service
metadata:
  name: svc-b3-liveness
  namespace: troubleshooting-lab
spec:
  selector:
    app: app-b3
  ports:
    - port: 80
      targetPort: 80
  ---
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-c1-dns-policy
  namespace: troubleshooting-lab
  labels:
    scenario: c1
spec:
  dnsPolicy: None
  dnsConfig:
    nameservers:
      - 1.2.3.4
  containers:
    - name: debug
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-c2-dns-debug
  namespace: troubleshooting-lab
  labels:
    scenario: c2
spec:
  containers:
    - name: debug
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
```

- **Paso 8.** Valida el manifiesto completo antes de crear los recursos, comprobando que los ocho escenarios son aceptados sintácticamente por Kubernetes.

```
kubectl apply --dry-run=client -f setup-
failures.yaml
```

 Copiar

Salida esperada:

La salida debe listar los Pods, Services y el Deployment como `created (dry run)` sin errores de validación.

- **Paso 9.** Aplica todos los escenarios y espera 30 segundos para que los reinicios, fallos de pull y eventos de scheduling sean observables.

```
kubectl apply -f setup-failures.yaml  
sleep 30
```

 Copiar

Salida esperada aproximada:

```
pod/pod-a1-imagepull created  
pod/pod-a2-crashloop created  
pod/pod-a3-pending created  
pod/pod-b1-backend created  
service/svc-b1-wrong-selector created  
pod/pod-b2-webserver created  
service/svc-b2-wrong-port created  
deployment.apps/deploy-b3-liveness created  
service/svc-b3-liveness created  
pod/pod-c1-dns-policy created  
pod/pod-c2-dns-debug created
```

 Copiar

- **Paso 10.** Obtén una vista inicial de Pods, Deployments, Services y eventos recientes.

```
kubectl get pods -n troubleshooting-lab  
kubectl get deployments -n troubleshooting-lab  
kubectl get services -n troubleshooting-lab  
kubectl get events -n troubleshooting-lab \  
  --sort-by='.lastTimestamp' | tail -30
```

 Copiar

Salida esperada aproximada de Pods:

NAME	READY	STATUS	 Copiar
pod-a1-imagepull	0/1		
ImagePullBackOff			
pod-a2-crashloop	0/1		
CrashLoopBackOff			
pod-a3-pending	0/1	Pending	
pod-b1-backend	1/1	Running	
pod-b2-webserver	1/1	Running	
deploy-b3-liveness-<hash>-<id>	...	Running	
deploy-b3-liveness-<hash>-<id>	...	Running	
pod-c1-dns-policy	1/1	Running	
pod-c2-dns-debug	1/1	Running	

A1 puede aparecer temporalmente como `ErrImagePull` y B3 puede alternar disponibilidad/reinicios debido a la liveness probe defectuosa.

IMPORTANTE: `ImagePullBackOff`, `CrashLoopBackOff` y `Pending` son resultados esperados en este punto. No intentes corregirlos todavía.

Tarea 1. Aplicar la metodología de troubleshooting

Antes de resolver escenarios individuales, utilizarás siempre el mismo flujo:

1. OBSERVAR → `kubectl get`
2. DESCRIBIR → `kubectl describe`
3. ANALIZAR → `kubectl logs` / `kubectl get events`
4. CORREGIR → `kubectl patch` / `apply` / `delete` + `recreate`
5. VALIDAR → estado + prueba funcional

Tarea 1.1. Construir una línea base

- **Paso 11.** Ejecuta una vista amplia de los recursos y registra qué escenarios muestran síntomas evidentes y cuáles requieren pruebas funcionales.

```
kubectl get pods \
  -n troubleshooting-lab \
  -o wide

kubectl get services \
  -n troubleshooting-lab

kubectl get events \
  -n troubleshooting-lab \
  --sort-by='.lastTimestamp'
```

 Copiar

Tarea 1.2. Priorizar evidencia

- **Paso 12.** Selecciona un Pod fallido y confirma que la sección **Events** de `kubectl describe` aporta más contexto que el campo STATUS aislado.

```
kubectl describe pod pod-a1-imagepull \
  -n troubleshooting-lab
```

 Copiar

Concepto clave: El estado visible describe el síntoma; Events, logs y configuración efectiva permiten acercarse a la causa raíz.

Tarea finalizada

Resultado esperado: Metodología sistemática de troubleshooting aplicada correctamente mediante observación, descripción, análisis, corrección y validación.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 1 con IA](#)

Tarea 2. Resolver fallos de Pods

En esta tarea corregirás tres fallos clásicos: imagen inexistente, proceso que termina inmediatamente y scheduling imposible.

Tarea 2.1. ImagePullBackOff

- **Paso 13.** Observa y describe `pod-a1-imagepull`, prestando atención a los eventos `Failed`, `ErrImagePull` y `BackOff`.

```
kubectl get pod pod-a1-imagepull \
-n troubleshooting-lab

kubectl describe pod pod-a1-imagepull \
-n troubleshooting-lab
```

[Copiar](#)

- **Paso 14.** Confirma qué imagen está configurada actualmente.

```
kubectl get pod pod-a1-imagepull \
-n troubleshooting-lab \
-o jsonpath='{.spec.containers[0].image}'
{"\n"}
```

[Copiar](#)

- **Paso 15.** Corrige el escenario recreando el Pod con una imagen existente.

```
kubectl delete pod pod-a1-imagepull \
-n troubleshooting-lab

kubectl run pod-a1-imagepull \
--image=nginx:1.27-alpine \
--labels=scenario=a1 \
-n troubleshooting-lab
```

[Copiar](#)

- **Paso 16.** Espera a que el Pod quede disponible.

```
kubectl wait \  
  --for=condition=Ready \  
  pod/pod-a1-imagepull \  
  -n troubleshooting-lab \  
  --timeout=90s
```

 Copiar

Tarea 2.2. CrashLoopBackOff

- **Paso 17.** Revisa `describe` y los logs de la ejecución anterior para identificar el comando inexistente.

```
kubectl describe pod pod-a2-crashloop \  
  -n troubleshooting-lab  
  
kubectl logs pod-a2-crashloop \  
  -n troubleshooting-lab \  
  --previous
```

 Copiar

- **Paso 18.** Consulta el comando configurado en el Pod y relaciónalo con el error de shell.

```
kubectl get pod pod-a2-crashloop \  
  -n troubleshooting-lab \  
  -o jsonpath='{.spec.containers[0].command}'  
{"\"n\"}"
```

 Copiar

- **Paso 19.** Recrea el Pod permitiendo que NGINX utilice su comando predeterminado.

```
kubectl delete pod pod-a2-crashloop \  
  -n troubleshooting-lab  
  
kubectl run pod-a2-crashloop \  
  --image=nginx:1.27-alpine \  
  --labels=scenario=a2 \  
  -n troubleshooting-lab
```

 Copiar

Tarea 2.3. Pending por recursos imposibles

- **Paso 20.** Describe `pod-a3-pending` y localiza el evento `FailedScheduling`.

```
kubectl describe pod pod-a3-pending \
-n troubleshooting-lab
```

 Copiar

- **Paso 21.** Consulta los requests definidos y confirma que ningún nodo del laboratorio puede satisfacerlos.

```
kubectl get pod pod-a3-pending \
-n troubleshooting-lab \
-o
jsonpath='{.spec.containers[0].resources.request
s}{"\n"}'
```

 Copiar

- **Paso 22.** Recrea el Pod con requests razonables para el entorno Minikube.

```
kubectl delete pod pod-a3-pending \
-n troubleshooting-lab
```

 Copiar

```
cat > pod-a3-fixed.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: pod-a3-pending
  namespace: troubleshooting-lab
  labels:
    scenario: a3
spec:
  containers:
  - name: app
    image: nginx:1.27-alpine
    resources:
      requests:
        cpu: 50m
        memory: 32Mi
      limits:
        cpu: 200m
        memory: 128Mi
EOF
```

 Copiar

```
kubectl apply -f pod-a3-fixed.yaml
```

 Copiar

```
kubectl wait --for=condition=Ready pod/pod-a3-
pending -n troubleshooting-lab --timeout=90s
```

 Copiar

Salida esperada:

```
pod "pod-a3-pending" deleted
pod/pod-a3-pending created
pod/pod-a3-pending condition met
```

 Copiar

- **Paso 23.** Verifica que los tres escenarios A1, A2 y A3 queden finalmente en **Running**.

```
kubect1 get pods \  
-n troubleshooting-lab \  
-l 'scenario in (a1,a2,a3)'
```

 Copiar

Tarea finalizada

Resultado esperado: Fallos `ImagePullBackOff`, `CrashLoopBackOff` y `Pending` diagnosticados y corregidos hasta recuperar Pods operativos.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 2 con IA](#)

Tarea 3. Resolver fallos de Services y probes

Ahora diagnosticarás tres escenarios donde los Pods pueden estar presentes, pero la conectividad o disponibilidad sigue fallando.

Tarea 3.1. Selector incorrecto

- **Paso 24.** Comprueba el Service y sus endpoints antes de modificar cualquier selector.

```
kubect1 get service svc-b1-wrong-selector \  
-n troubleshooting-lab
```

 Copiar

```
kubect1 get endpoints svc-b1-wrong-selector \  
-n troubleshooting-lab
```

 Copiar

- **Paso 25.** Compara el selector del Service con los labels del Pod destino.

```
kubect1 describe service svc-b1-wrong-selector \  
-n troubleshooting-lab
```

 Copiar

```
kubectl get pod pod-b1-backend \
  -n troubleshooting-lab \
  --show-labels
```

 Copiar

- **Paso 26.** Corrige únicamente el selector incorrecto.

```
kubectl patch service svc-b1-wrong-selector \
  -n troubleshooting-lab \
  --type=merge \
  -p '{"spec":{"selector":{"app":"backend-
real"}}}'
```

 Copiar

- **Paso 27.** Comprueba que los endpoints aparecen automáticamente.

```
kubectl get endpoints svc-b1-wrong-selector \
  -n troubleshooting-lab
```

 Copiar

Tarea 3.2. targetPort incorrecto

- **Paso 28.** Comprueba que `svc-b2-wrong-port` sí tiene endpoint, pero observa el puerto asociado.

```
kubectl get endpoints svc-b2-wrong-port \
  -n troubleshooting-lab
```

 Copiar

```
kubectl describe service svc-b2-wrong-port \
  -n troubleshooting-lab
```

 Copiar

- **Paso 29.** Confirma que NGINX escucha en el puerto 80 según la configuración del Pod.

```
kubectl get pod pod-b2-webserver \
  -n troubleshooting-lab \
  -o
jsonpath='{.spec.containers[0].ports[0].containerPort}'
```

 Copiar

- **Paso 30.** Corrige `targetPort` para dirigir tráfico al puerto correcto.

```
kubectl patch service svc-b2-wrong-port \
  -n troubleshooting-lab \
  --type=merge \
  -p '{"spec":{"ports":
[{"port":80,"targetPort":80}]}}'
```

 Copiar

Tarea 3.3. Liveness probe incorrecta

- **Paso 31.** Obtén uno de los Pods del Deployment y revisa Events para localizar mensajes `Unhealthy`.

```
POD_B3=$(kubectl get pod \
  -n troubleshooting-lab \
  -l app=app-b3 \
  -o jsonpath='{.items[0].metadata.name}')

kubectl describe pod "$POD_B3" \
  -n troubleshooting-lab
```

 Copiar

- **Paso 32.** Consulta la ruta configurada en la liveness probe.

```
kubectl get deployment deploy-b3-liveness \
  -n troubleshooting-lab \
  -o
jsonpath='{.spec.template.spec.containers[0].livenessProbe.httpGet.path}'
```

 Copiar

- **Paso 33.** Corrige la ruta a `/`, provocando un nuevo rollout del Deployment.

```
kubectl patch deployment deploy-b3-liveness \
  -n troubleshooting-lab \
  --type='json' \
  -
p='[{"op":"replace","path":"/spec/template/spec/containers/0/livenessProbe/httpGet/path","value":"/"}]'
```

 Copiar

- **Paso 34.** Espera a que el Deployment alcance nuevamente disponibilidad completa.

```
kubectl rollout status \  
  deployment/deploy-b3-liveness \  
-n troubleshooting-lab \  
--timeout=120s
```

[Copiar](#)

- **Paso 35.** Confirma que los tres Services tienen destinos disponibles.

```
kubectl get endpoints \  
-n troubleshooting-lab \  
svc-b1-wrong-selector \  
svc-b2-wrong-port \  
svc-b3-liveness
```

[Copiar](#)

Tarea finalizada

Resultado esperado: Problemas de selector, `targetPort` y liveness probe corregidos, restaurando endpoints y conectividad de los Services.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 3 con IA](#)

Tarea 4. Diagnosticar fallos de DNS

En esta tarea resolverás una configuración DNS incorrecta y luego verificarás la salud normal de CoreDNS desde un Pod de diagnóstico.

Tarea 4.1. dnsPolicy incorrecta

- **Paso 36.** Comprueba que `pod-c1-dns-policy` está `Running`; este estado demuestra que un Pod puede estar sano a nivel de proceso y fallar únicamente en resolución DNS.

```
kubectl get pod pod-c1-dns-policy \
-n troubleshooting-lab
```

 Copiar

Salida esperada aproximada:

NAME	READY	STATUS
pod-c1-dns-policy	1/1	Running

 Copiar

- **Paso 37.** Ejecuta una consulta acotada al FQDN de `kubernetes.default`; el timeout evita que una configuración DNS inválida mantenga la terminal esperando demasiado tiempo.

```
kubectl exec pod-c1-dns-policy \
-n troubleshooting-lab -- \
sh -c 'timeout 5 nslookup
kubernetes.default.svc.cluster.local || echo
"DNS_FAILED"'
```

 Copiar

Salida esperada durante el fallo:

```
DNS_FAILED
```

 Copiar

Puede aparecer además un mensaje de timeout generado por `nslookup`.

- **Paso 38.** Revisa `/etc/resolv.conf` dentro del contenedor utilizando `sh -c` para evitar que Git Bash transforme la ruta Linux.

```
kubectl exec pod-c1-dns-policy \
-n troubleshooting-lab -- \
sh -c 'cat /etc/resolv.conf'
```

 Copiar

Salida esperada aproximada:

```
nameserver 1.2.3.4
```

 Copiar

- **Paso 39.** Confirma mediante la API que el Pod utiliza `dnsPolicy: None` y una configuración DNS explícita.

```
kubectl get pod pod-c1-dns-policy \
-n troubleshooting-lab \
-o jsonpath='{.spec.dnsPolicy}{"\n"}'
```

 Copiar

```
kubectl get pod pod-c1-dns-policy \
-n troubleshooting-lab \
-o jsonpath='{.spec.dnsConfig}{"\n"}'
```

 Copiar

Salida esperada aproximada:

```
None
{"nameservers":["1.2.3.4"]}
```

 Copiar

- **Paso 40.** Recrea únicamente el Pod C1 con `ClusterFirst`, retirando la configuración personalizada sin modificar CoreDNS ni el Service `kube-dns`.

```
kubectl delete pod pod-c1-dns-policy \
-n troubleshooting-lab
```

 Copiar

```
cat > pod-c1-fixed.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: pod-c1-dns-policy
  namespace: troubleshooting-lab
  labels:
    scenario: c1
spec:
  dnsPolicy: ClusterFirst
  containers:
    - name: debug
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
EOF
```

 Copiar

```
kubectl apply -f pod-c1-fixed.yaml
```

 Copiar

```
kubectl wait \
  --for=condition=Ready \
  pod/pod-c1-dns-policy \
  -n troubleshooting-lab \
  --timeout=60s
```

 Copiar

Salida esperada:

```
pod "pod-c1-dns-policy" deleted
pod/pod-c1-dns-policy created
pod/pod-c1-dns-policy condition met
```

 Copiar

- **Paso 41.** Repite la consulta usando el FQDN completo para validar de forma determinista que el Pod vuelve a consultar el DNS del clúster.

```
kubectl exec pod-c1-dns-policy \
  -n troubleshooting-lab -- \
  nslookup kubernetes.default.svc.cluster.local
```

 Copiar

Salida esperada aproximada:

```
Name:      kubernetes.default.svc.cluster.local
Address: 10.x.x.x
```

 Copiar

Tarea 4.2. Diagnóstico de CoreDNS

- **Paso 42.** Revisa los Pods y el Service de CoreDNS.

```
kubectl get pods \
  -n kube-system \
  -l k8s-app=kube-dns

kubectl get service kube-dns \
  -n kube-system
```

 Copiar

- **Paso 43.** Comprueba que el Service `kube-dns` dispone de endpoints activos.

```
kubectl get endpoints kube-dns \
-n kube-system
```

 Copiar

- **Paso 44.** Utiliza `pod-c2-dns-debug` para resolver un Service del namespace y el Service `kubernetes.default`.

```
kubectl exec pod-c2-dns-debug \
-n troubleshooting-lab -- \
nslookup svc-b1-wrong-selector
```

 Copiar

```
kubectl exec pod-c2-dns-debug \
-n troubleshooting-lab -- \
nslookup kubernetes.default.svc.cluster.local
```

 Copiar

- **Paso 45.** Revisa `/etc/resolv.conf` y relaciona `nameserver` y `search` con el comportamiento observado.

```
kubectl exec pod-c2-dns-debug \
-n troubleshooting-lab -- \
sh -c 'cat /etc/resolv.conf'
```

 Copiar

NOTA: No es necesario modificar CoreDNS para completar este escenario. El objetivo es demostrar que el servicio DNS del clúster está sano después de corregir la política DNS del Pod C1.

Tarea finalizada

Resultado esperado: Fallos de DNS diagnosticados y corregidos mediante `dnsPolicy`, resolución interna y validación del funcionamiento de CoreDNS.

Prompt de apoyo para explicar lo realizado

➤ [Explicar la Tarea 4 con IA](#)

✓ Tarea 5. Ejecutar validación integral

La última tarea verifica que los ocho escenarios se encuentran corregidos y que las pruebas funcionales básicas responden.

Tarea 5.1. Validar Pods y Deployment

- **Paso 46.** Ejecuta las comprobaciones siguientes.

```
kubectl get pods \  
-n troubleshooting-lab
```

 Copiar

```
kubectl get deployment deploy-b3-liveness \  
-n troubleshooting-lab
```

Tarea 5.2. Validar Services

- **Paso 47.** Crea un Pod temporal para probar los tres Services corregidos.

```
kubectl run final-test \  
--rm -i \  
--restart=Never \  
--image=busybox:1.36 \  
-n troubleshooting-lab \  
-- sh -c '  
  echo "=== B1 ==="  
  wget -qO- http://svc-b1-wrong-selector |  
head -2  
  
  echo "=== B2 ==="  
  wget -qO- http://svc-b2-wrong-port | head -2  
  
  echo "=== B3 ==="  
  wget -qO- http://svc-b3-liveness | head -2  
'
```

 Copiar

Tarea 5.3. Ejecutar el resumen automático

- **Paso 48.** Ejecuta el bloque siguiente para verificar estados, correcciones de Service, liveness probe, endpoints y DNS mediante FQDN completos.

```
echo "=== Validacion final de la Practica 8 ==="
```

[Copiar](#)

```
for POD in \
    pod-a1-imagepull \
    pod-a2-crashloop \
    pod-a3-pending \
    pod-b1-backend \
    pod-b2-webserver \
    pod-c1-dns-policy \
    pod-c2-dns-debug
do
    STATUS=$(kubectl get pod "$POD" \
        -n troubleshooting-lab \
        -o jsonpath='{.status.phase}')

    echo "$POD: $STATUS"
done

READY=$(kubectl get deployment deploy-b3-
liveness \
    -n troubleshooting-lab \
    -o jsonpath='{.status.readyReplicas}')

echo "deploy-b3-liveness ready: ${READY:-0}/2"

B1_SELECTOR=$(kubectl get service svc-b1-wrong-
selector \
    -n troubleshooting-lab \
    -o jsonpath='{.spec.selector.app}')

B2_TARGET=$(kubectl get service svc-b2-wrong-
port \
    -n troubleshooting-lab \
    -o jsonpath='{.spec.ports[0].targetPort}')

B3_PATH=$(kubectl get deployment deploy-b3-
liveness \
    -n troubleshooting-lab \
    -o
    jsonpath='{.spec.template.spec.containers[0].liv
enessProbe.httpGet.path}')

echo "B1 selector: $B1_SELECTOR"
echo "B2 targetPort: $B2_TARGET"
```

```
echo "B3 liveness path: $B3_PATH"

echo ""
echo "Endpoints:"

kubectl get endpoints \
  -n troubleshooting-lab \
  svc-b1-wrong-selector \
  svc-b2-wrong-port \
  svc-b3-liveness

echo ""
echo "DNS:"

kubectl exec pod-c1-dns-policy \
  -n troubleshooting-lab -- \
  nslookup kubernetes.default.svc.cluster.local
>/dev/null 2>&1 \
  && echo "C1 DNS: OK" \
  || echo "C1 DNS: FAIL"

kubectl exec pod-c2-dns-debug \
  -n troubleshooting-lab -- \
  nslookup svc-b1-wrong-
selector.troubleshooting-lab.svc.cluster.local
>/dev/null 2>&1 \
  && echo "C2 DNS: OK" \
  || echo "C2 DNS: FAIL"

echo "=== Fin de validacion ==="
```

Salida esperada aproximada:

```
=== Validacion final de la Practica 8 ===  
pod-a1-imagepull: Running  
pod-a2-crashloop: Running  
pod-a3-pending: Running  
pod-b1-backend: Running  
pod-b2-webserver: Running  
pod-c1-dns-policy: Running  
pod-c2-dns-debug: Running  
deploy-b3-liveness ready: 2/2  
B1 selector: backend-real  
B2 targetPort: 80  
B3 liveness path: /  
  
Endpoints:  
svc-b1-wrong-selector    10.244.x.x:80  
svc-b2-wrong-port        10.244.x.x:80  
svc-b3-liveness          10.244.x.x:80,10.244.x.x:80  
  
DNS:  
C1 DNS: OK  
C2 DNS: OK  
=== Fin de validacion ===
```

 Copiar

Tarea finalizada

Resultado esperado: Ocho escenarios de fallo validados satisfactoriamente, confirmando Pods, Services, endpoints y resolución DNS operativos.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 5 con IA](#)

Resolución de problemas

Problema 1. ImagePullBackOff continúa después de corregir la imagen

Síntoma: El Pod sigue sin iniciar aunque la imagen parece válida.

Causa probable: Existe un problema de conectividad, rate limiting del registro o la imagen no está disponible para la arquitectura utilizada.

Solución:

```
kubectl describe pod pod-a1-imagepull \  
-n troubleshooting-lab
```

[Copiar](#)

Lee el mensaje exacto de Events antes de cambiar nuevamente el manifiesto.

Problema 2. logs --previous no devuelve información

Síntoma: `kubectl logs --previous` indica que no existe una instancia anterior del contenedor.

Causa probable: El contenedor todavía no ha reiniciado o el Pod fue recreado recientemente.

Solución:

```
kubectl get pod pod-a2-crashloop \  
-n troubleshooting-lab
```

[Copiar](#)

```
kubectl describe pod pod-a2-crashloop \  
-n troubleshooting-lab
```

Utiliza logs normales si existe una instancia activa o espera a que ocurra al menos un reinicio.

Problema 3. El Pod A3 continúa Pending

Síntoma: El Pod sigue sin ser programado después de reducir requests.

Causa probable: Existe otro motivo de scheduling distinto a CPU o memoria.

Solución:

```
kubectl describe pod pod-a3-pending \  
-n troubleshooting-lab
```

[Copiar](#)

Lee el evento `FailedScheduling` completo. Puede señalar taints, affinity o restricciones de nodos.

Problema 4. Un Service tiene endpoints pero wget falla

Síntoma: El Service muestra destinos activos, pero la conexión HTTP no funciona.

Causa probable: El puerto destino es incorrecto o la aplicación no escucha donde espera el Service.

Solución:

```
kubectl describe service svc-b2-wrong-port \
-n troubleshooting-lab

kubectl get pod pod-b2-webserver \
-n troubleshooting-lab \
-o yaml
```

 Copiar

Compara `targetPort` con el puerto real de la aplicación.

Problema 5. DNS interno sigue sin resolver

Síntoma: El Pod usa `ClusterFirst`, pero `nslookup` continúa fallando.

Causa probable: CoreDNS no está disponible o el Service `kube-dns` no tiene endpoints.

Solución:

```
kubectl get pods \
-n kube-system \
-l k8s-app=kube-dns

kubectl get endpoints kube-dns \
-n kube-system

kubectl exec pod-c1-dns-policy \
-n troubleshooting-lab -- \
cat /etc/resolv.conf
```

 Copiar

No modifiques CoreDNS hasta comprobar primero estos tres elementos.

Problema 6. nslookup muestra una respuesta válida y termina con exit code 1

Síntoma: BusyBox muestra la dirección correcta para un Service, pero también imprime consultas `NXDOMAIN` y `kubectl exec` termina con código `1`.

Causa probable: `nslookup` intenta varias expansiones definidas por los search domains y alguna de ellas falla aunque otra haya resuelto correctamente.

Solución:

Para validaciones automáticas utiliza el FQDN completo:

```
kubectl exec pod-c2-dns-debug \
  -n troubleshooting-lab -- \
  nslookup svc-b1-wrong-selector.troubleshooting-
  lab.svc.cluster.local
```

[Copiar](#)

Utiliza el nombre corto en pruebas funcionales HTTP cuando quieras demostrar el efecto del search domain.

Problema 7. Git Bash transforma /etc/resolv.conf

Síntoma: El comando intenta acceder a una ruta parecida a `C:/Program Files/Git/etc/resolv.conf`.

Causa probable: MSYS2 convirtió la ruta Linux antes de entregarla a `kubectl`.

Solución:

```
kubectl exec pod-c2-dns-debug \
  -n troubleshooting-lab -- \
  sh -c 'cat /etc/resolv.conf'
```

[Copiar](#)

Problema 8. final-test permanece esperando

Síntoma: La prueba funcional de Services tarda demasiado en devolver resultado.

Causa probable: Algún Service todavía apunta a un destino incorrecto o carece de endpoints.

Solución:

Los `wget` de la práctica utilizan `-T 5` para limitar la espera. Revisa:

```
kubectl get endpoints -n troubleshooting-lab  
kubectl get services -n troubleshooting-lab
```

[Copiar](#)

Identifica qué escenario permanece incorrecto antes de repetir la validación.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar la práctica habrás diagnosticado y corregido ocho escenarios de fallo utilizando un flujo sistemático basado en observación, descripción, análisis, corrección y validación funcional.

Notas y consideraciones

- Todos los escenarios se despliegan dentro del namespace `troubleshooting-lab`.
- Los errores iniciales son intencionales y forman parte del ejercicio.
- Evita corregir recursos antes de revisar Events, logs o configuración efectiva.
- No modifiques CoreDNS; los escenarios DNS se resuelven desde la configuración de los Pods.
- En Git Bash, las rutas Linux dentro de contenedores se consultan mediante `sh -c` para evitar conversiones automáticas de MSYS2.
- Para validaciones DNS deterministas con BusyBox se utiliza el FQDN completo; `nslookup` sobre nombres cortos puede mostrar respuestas válidas y aun así terminar con código distinto de cero por intentos adicionales del search domain.

URL de referencia

- [Debugging Pods](#)
- [Debugging Services](#)
- [Debugging DNS Resolution](#)
- [kubectl describe](#)